Visualization of Kerr Black Hole Based on Ray Tracing

1 Introduction

In general relativity, black holes are regions of extreme spacetime curvature where the gravitational field is so strong that even light cannot escape. To visualize the gravitational effects of black holes, we employ ray tracing techniques to simulate the propagation paths of light rays around black holes. This paper will introduce how to implement this process using C++ and OpenGL 3.3, demonstrating both Newtonian gravitational approximation and real simulation methods based on general relativity.

2 How to Implement Ray Tracing with CGP and OpenGL 3.3

We use CGP as the graphics library interface and OpenGL 3.3 for rendering. The core concept of ray tracing is to trace rays from the camera along the direction of each pixel, calculating their interactions with black holes and other celestial bodies.

The ray tracing implementation is divided into two main parts: the ray tracing core algorithm and the rendering module. The ray tracing core algorithm is responsible for calculating the propagation paths of light rays in the black hole's gravitational field, while the rendering module is responsible for visualizing the calculation results.

The ray tracing implementation requires the following steps:

1. Define the ray class, including the ray's origin and direction.
2. Implement the ray tracing core algorithm to calculate the propagation paths of light rays in the black hole's gravitational field.
3. Implement the accretion disk rendering model to simulate the luminous plasma structures around the black hole.
4. Implement the planetary orbit simulation module to display planetary orbits near the black hole.
5. Implement the background rendering module to render the starry sky and virtual ground in the distance from the black hole.

The visualization implementation has two modules: Newtonian gravitational approximation and real simulation based on general relativity.

3 Newtonian Gravitational Approximation

3.1 Ray Tracing Core Algorithm

A ray `Ray` is represented by two `vec3` vectors:

1. `origin` represents the current position of the ray

2. `Direction` represents the ray direction

For light propagation in curved space, we use the following assumptions and dynamical model:

1. The ray's trajectory is only bent by the influence of the black hole (i.e., we do not consider the effects of other celestial bodies on the ray)
2. Light rays propagate in straight lines in front of the screen
3. After a certain period of propagation, light rays directly fall into infinity

During propagation, the ray satisfies the equation:

$$\boxed{\begin{cases} \dfrac{d\vec{x}}{dt} = \vec{v}(t) \\ \vec{a} = \dfrac{d\vec{v}}{dt} = -\dfrac{3}{2} \cdot \dfrac{|\vec{x} \times \vec{v}|^2 \vec{x}}{|\vec{x}|^5} + \dfrac{2a}{|\vec{x}|^3} (\vec{v} \times \hat{z}) \end{cases}} \tag{1}$$

where $\vec{x}$ is the ray's position, $\vec{v}$ is the ray's velocity, $\hat{z}$ is the black hole's spin axis direction, and a is the black hole's spin parameter. When the whitehole option is checked, the first term in the second equation takes the opposite sign. Initially, we default a to 0, meaning the black hole is non-spinning, but a can be controlled through the GUI.

We use three numerical computation methods to calculate light propagation: RK4, Verlet integration, and symplectic Euler method.

3.1.1 Runge-Kutta Fourth-Order Method (RK4)

RK4 is a fourth-order explicit method. Each step update is:

$$\vec{k}_1^v = \vec{a}(\vec{x}, \vec{v}) \tag{2}$$
$$\vec{k}_1^x = \vec{v} \tag{3}$$
$$\vec{k}_2^v = \vec{a}(\vec{x} + \tfrac{1}{2}\Delta t \cdot \vec{k}_1^x,\ \vec{v} + \tfrac{1}{2}\Delta t \cdot \vec{k}_1^v) \tag{4}$$
$$\vec{k}_2^x = \vec{v} + \tfrac{1}{2}\Delta t \cdot \vec{k}_1^v \tag{5}$$
$$\vec{k}_3^v = \vec{a}(\vec{x} + \tfrac{1}{2}\Delta t \cdot \vec{k}_2^x,\ \vec{v} + \tfrac{1}{2}\Delta t \cdot \vec{k}_2^v) \tag{6}$$
$$\vec{k}_3^x = \vec{v} + \tfrac{1}{2}\Delta t \cdot \vec{k}_2^v \tag{7}$$
$$\vec{k}_4^v = \vec{a}(\vec{x} + \Delta t \cdot \vec{k}_3^x,\ \vec{v} + \Delta t \cdot \vec{k}_3^v) \tag{8}$$
$$\vec{k}_4^x = \vec{v} + \Delta t \cdot \vec{k}_3^v \tag{9}$$
$$\vec{x} \leftarrow \vec{x} + \frac{\Delta t}{6}(\vec{k}_1^x + 2\vec{k}_2^x + 2\vec{k}_3^x + \vec{k}_4^x) \tag{10}$$
$$\vec{v} \leftarrow \vec{v} + \frac{\Delta t}{6}(\vec{k}_1^v + 2\vec{k}_2^v + 2\vec{k}_3^v + \vec{k}_4^v) \tag{11}$$

3.1.2 Verlet Integration

Verlet is a second-order integration method with the following steps:

$$\vec{a}_0 = \vec{a}(\vec{x}, \vec{v}) \tag{12}$$
$$\vec{x} \leftarrow \vec{x} + \vec{v}\Delta t + \tfrac{1}{2}\vec{a}_0 \Delta t^2 \tag{13}$$
$$\vec{a}_1 = \vec{a}(\vec{x}, \vec{v}) \tag{14}$$
$$\vec{v} \leftarrow \vec{v} + \tfrac{1}{2}(\vec{a}_0 + \vec{a}_1)\Delta t \tag{15}$$

3.1.3 Symplectic Euler Method

Symplectic Euler is the most basic symplectic method with the following structure:

$$\vec{v} \leftarrow \vec{v} + \Delta t \cdot \vec{a}(\vec{x}, \vec{v}) \tag{16}$$

$$\vec{x} \leftarrow \vec{x} + \Delta t \cdot \vec{v} \tag{17}$$

3.1.4 Method Comparison Table

Method	Cost per Step	Global Error	Conservation	Stability	Real-time Usability
Runge-Kutta 4	4 acceleration calculations	$O(\Delta t^4)$	×	✓	Medium
Verlet	2 acceleration calculations	$O(\Delta t^2)$	✓	✓	Medium
Symplectic Euler	1 acceleration calculation	$O(\Delta t)$	✓	Medium	✓

Table 1: Comparison of numerical integration methods

3.2 Ray Tracing Implementation Framework

Several camera parameters are passed from the CPU, and after processing, we can obtain the ray from any point on the screen.

`vec3 raytrace(Ray ray)`

- **input:** direction from a pixel on the screen relative to the observer, the actual spatial position of the pixel as origin
- **output:** the final color of that pixel.
- We initialize the color vector to black and the physical quantities corresponding to the ray
- We use numerical methods to step-by-step calculate the ray's direction and position at each small time moment dt and update the state.
- **Ray absorption:** If the ray enters the event horizon, return the current color
- **Accretion disk rendering:** If the ray "passes through" the accretion disk, add the accretion disk emission to the accumulated color based on the ray's position, direction, and accretion disk parameters.
- **Planet display:** If the ray hits a planet, add the planet's color to the accumulated color based on the planet's parameters and return.
- **Ray propagation to infinity:** After a specified number of steps, exit the loop, add background to the accumulated color, and return.

3.3 Accretion Disk Rendering Model

To achieve realistic accretion disk visualization in black hole rendering, we designed and implemented the `diskRender` function to simulate the luminous plasma structures rotating around the black hole. This module integrates physical models, procedural texture generation, and relativistic effects, ultimately generating dynamic luminous patterns in the fragment shader.

3.3.1 Geometric Modeling and Density Distribution

The accretion disk is modeled as a flat body distributed on the $y = 0$ plane, with density controlled by functions in vertical and radial directions:

- Disk height is controlled by `diskHeight`, with vertical density decay exponent `diskDensityV`;
- Radial range is defined by `innerRadius` and `outerRadius`;
- Horizontal density decay is constrained by the power function $\propto r^{-\alpha}$, where $\alpha = \text{diskDensityH}$;
- All positions are converted to spherical coordinates `sphericalCoord` for sampling procedural noise.

3.3.2 Procedural Noise Perturbation

We add dynamic perturbations to the accretion disk to simulate the inhomogeneity of hot gas:

- Uses fractal recursive Simplex noise $\text{snoise}(\cdot)$;
- Each noise level incorporates different time factors to simulate disk rotation and turbulent flow;
- Noise levels are controlled by `diskNoiseLOD`, adjustable at runtime.

3.3.3 Relativistic Effects Modeling

Light emitted from the accretion disk is affected by the following two key frequency shift effects during propagation:

Gravitational Redshift Using the approximate formula in Schwarzschild spacetime:

$$z = \sqrt{1 - \frac{r_s}{r}} - 1 \tag{18}$$

Manifesting in coloring as temperature redshift (dimming and reddening):

$$T \leftarrow \frac{T}{1+z} \tag{19}$$

Line-of-sight Doppler Shift Based on the accretion disk circular orbital velocity $v = \sqrt{\frac{1-3/r}{r}}$, combined with the observation direction, calculate the relativistic Doppler factor:

$$D = \gamma(1 + \vec{v} \cdot \hat{n}) \tag{20}$$

Final temperature correction:

$$T \leftarrow \frac{T}{D} \tag{21}$$

and multiply by brightness factor $\propto D^3$ to simulate relativistic luminosity boosting.

3.3.4 Black Body Color Mapping

Local disk temperature is mapped to RGB values through the `getBlackBodyColor(temp)` function:

- Temperature range is limited to $[1000K, 12000K]$;
- Uses `smoothstep` to construct continuous color bands from red to blue-white;
- Brightness is enhanced by $\sqrt{T}$, approximating the StefanBoltzmann radiation law.

3.3.5 Rendering Formula Summary

Final color values are superimposed as follows:

$$\text{color} \mathrel{+}= \rho(x) \cdot \alpha \cdot \text{noise}(x) \cdot C(T_{\text{eff}}(x)) \tag{22}$$

where:

- $\rho(x)$ is local density;
- $C(T)$ is the black body color function;
- $T_{\text{eff}}(x)$ includes frequency shift corrections;
- If `temperature_realistic` is enabled, then $T \propto r^{-3/4}$.

3.3.6 Configurable Parameters

Parameter Name	Meaning
`accretionTemp`	Accretion disk base temperature
`temperature_realistic`	Whether to enable realistic temperature decay
`red_shift`	Whether to enable gravitational redshift
`display_accretion_disk`	Whether to use relativistic velocity model
`diskNoiseLOD`	Noise levels
`diskDensityV/H`	Vertical/radial density control exponents

Table 2: Configurable parameters for accretion disk rendering

This model provides a good balance between visual realism and performance, suitable for dynamically simulating brightness fluctuations, color transitions, and relativistic beaming visual effects of accretion disks.

3.4 Planetary Orbit Simulation Module

This module models and renders planetary motion near black holes through classical celestial mechanics, displaying their time-varying orbital positions and surface grid lines. This section includes three main parts: Kepler equation solving, orbital parameter transformation, and geographic coordinate coloring.

3.4.1 Planetary Orbital Modeling

Planetary positions are calculated using six classical orbital elements (Keplerian elements):

Parameter Name	Physical Meaning
`A`	Semi-major axis (demi_grand_axe)
`e`	Eccentricity
`i`	Orbital inclination
Ω	Longitude of ascending node (longtitude_du_noeud_ascendant)
ω	Argument of periapsis (argument_du_periapside)
M_0	Initial mean anomaly (mean_anomaly)

Table 3: Keplerian orbital elements

The orbital position of a planet at any time t is obtained through the following steps:

1. Calculate current mean anomaly: $M = M_0 + nt$, where $n = \sqrt{1/A^3}$ is the mean motion angular velocity;
2. Use Newton-Raphson method to iteratively solve Kepler's equation:

$$E - e \sin E = M \tag{23}$$

3. Use eccentric anomaly E to solve for true anomaly ν and orbital radius r;
4. Transform from orbital coordinate system (x, y) to three-dimensional space coordinates (X, Y, Z), considering inclination and rotation.

This process is implemented through `computePlanetPosition(t)`.

3.4.2 Kepler Equation Solver

Function `solveKepler(M, e)` uses five Newton-Raphson iterations to quickly approximate the solution for E:

$$f(E) = E - e \sin E - M \tag{24}$$

$$f'(E) = 1 - e \cos E \tag{25}$$

$$E_{n+1} = E_n - \frac{f(E_n)}{f'(E_n)} \tag{26}$$

Since $e < 1$ and M is bounded, 5 iterations are sufficient for visualization accuracy requirements.

3.4.3 Orbital Position Output

Final output of planetary position in three-dimensional space:

$$\vec{r}(t) = \begin{bmatrix} r(\cos\Omega\cos(\omega+\nu) - \sin\Omega\sin(\omega+\nu)\cos i) \\ r(\sin\Omega\cos(\omega+\nu) + \cos\Omega\sin(\omega+\nu)\cos i) \\ r\sin(\omega+\nu)\sin i \end{bmatrix} \tag{27}$$

The code returns `vec3(X, Z, Y)` to adapt to the right-handed coordinate system.

3.4.4 Grid Texture Coloring

Function `planet_grid_color(...)` simulates planetary surface longitude and latitude lines:

1. Convert viewing direction `pos - planet_pos` to spherical coordinates (θ, ϕ);
2. Use modulo operations and `step` function to determine proximity to meridians or parallels;
3. Output parameter `grid` is used for overlay grid line rendering.

In default settings, grid line spacing is 30° with line width of 2°.

While this module does not consider gravitational bending and other relativistic effects in rendering, its orbital solution is completely based on Keplerian mechanics, suitable for representing stable celestial body motion characteristics at far distances from black holes.

3.5 Background Rendering Module

This module is responsible for rendering two types of visual backgrounds: the distant starry sky around the black hole and the virtual checkerboard ground. In ray tracing, if a ray is not absorbed by the black hole or other objects, the final background color contribution is determined based on its direction. There are two types of backgrounds:

1. Spherical Sky background,
2. Virtual ground background (Plane).

3.5.1 Spherical Sky Background Rendering

Function `background_sky_sphere(direction)` uses unit sphere mapping to map any direction vector `direction` to UV coordinates in the $[0, 1]^2$ range:

$$u = 0.5 + \frac{\arctan(x, y)}{2\pi} \tag{28}$$

$$v = \frac{\arccos(-z)}{\pi} \tag{29}$$

Then scale according to `background_distance`, pass to star map function `star_map_uv(uv)` to obtain corresponding intensity, then use `skyColor(...)` to get actual RGB values.

3.5.2 Star Map Sampling (`star_map_uv`)

This function is the core of procedurally generating star map noise, with the following steps:

1. Quantize UV coordinates to 512 pixel grid;
2. Sample hash function for coordinates at different frequencies;
3. Use `smoothstep` and `mix` to control star density and probability distribution, density can be controlled through `background_scale`.

Star map intensity is defined as:

$$\text{star} = \text{smoothstep}(\text{threshold}, 1.0, s_1) \cdot s_2 \tag{30}$$

3.5.3 Sky Color Palette (`getSkyPaletteColor`)

Defines typical colors for five main sequence star types:

Type	Color	Description
M	(1.0, 0.5, 0.3)	Red giant
K	(1.0, 0.7, 0.4)	Orange star
G	(1.0, 0.95, 0.7)	Yellow main sequence (sun-like)
F	(1.0, 1.0, 0.9)	White star
A	(0.9, 0.95, 1.0)	Blue-white star

Table 4: Stellar color classification

Star colors are interpolated in the color band with noise perturbation and intensity t, simulating stellar color dispersion.

3.5.4 Brightness Adjustment (`skyColor`)

Brightness and contrast are adjusted as follows:

$$\text{brightness} = (\text{starIntensity} \cdot 2.0)^{0.8} \tag{31}$$

Color modulation formula:

$$\text{color} = \text{baseColor} \cdot \text{brightness} \tag{32}$$

3.5.5 Virtual Ground Background (Plane)

When `background_type != 2`, use `background_plane(...)` to render checkerboard or virtual plane:

- Only hits when ray is directed downward;
- If checkerboard mode is enabled, alternately mix 0.2 and 0.95 brightness in checkerboard pattern;
- Otherwise generate color gradients directly using coordinates.

3.5.6 Background Mode Selection

Main function `background_color(x, direction)` determines which background to use: by adjusting `background_type`, you can switch between celestial star maps and virtual planes. Parameters like `background_distance`, `background_scale`, etc., allow users to flexibly control background detail representation.

4 Real Simulation Based on General Relativity

This algorithm is based on Section 2.1 (Basic Concepts) and Appendix A.1 (Ray Tracing) of arXiv:1502.03808v2. We reproduced this algorithm, but for unknown reasons, we were unable to accurately reproduce the black hole visualization phenomena.

This algorithm requires as input the camera position (r_c, θ_c, ϕ_c) in the FIDO coordinate system, the camera's velocity β relative to the FIDO coordinate system and velocity direction vector (B_r, B_θ, B_ϕ), as well as the incident ray direction relative to the camera (θ_{cs}, ϕ_{cs}). After initialization, the light ray propagation trajectory can be obtained similarly.

We conjecture that the reasons for the algorithm's failure are as follows:

1. Instability inherent in the numerical solution itself
2. During the algorithm implementation process, we did not fully understand the algorithm details, leading to implementation deviations
3. The algorithm implementation requires higher precision numerical calculations, while the numerical methods we used may not be sufficiently accurate

5 Conclusion

This paper introduces how to implement Kerr black hole visualization based on ray tracing using C++ and OpenGL 3.3. We implemented two methods: Newtonian gravitational approximation and real simulation based on general relativity. Through the ray tracing core algorithm, accretion disk rendering model, planetary orbit simulation module, and background rendering module, we successfully demonstrated the complex physical phenomena of black holes and their surrounding environments.

6 References

1. arXiv:1502.03808v2 - Black hole visualization algorithm based on general relativity.
2. MTW, “Gravitation” - Classic textbook on general relativity.